POLITECNICO
DI MILANO

www.polimi.it

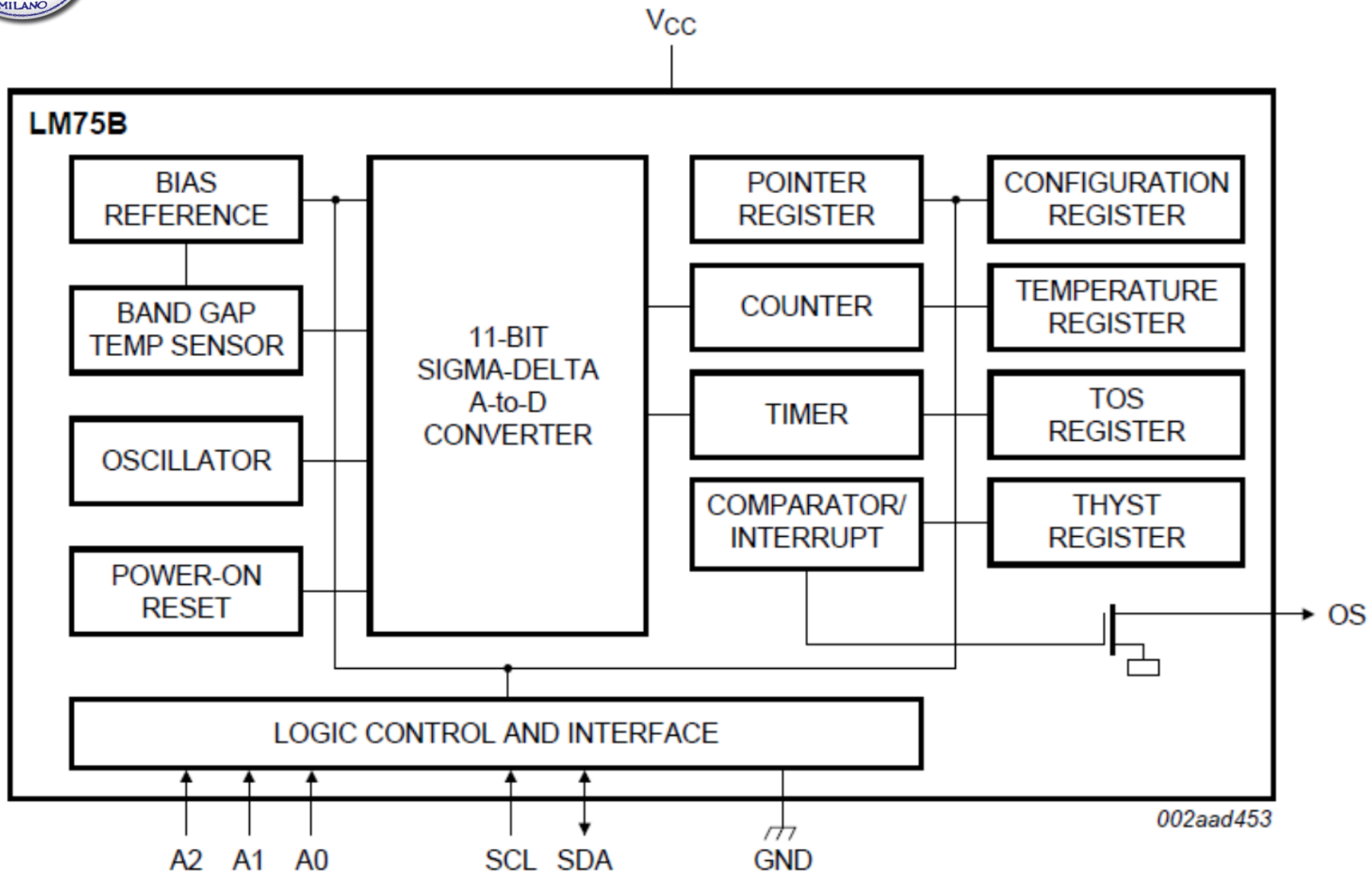




STM32 – Temperature sensor

Ing. Arianna De Vecchi, Prof. Federica Villa

Temperature sensor on POLIMI board



I^2C interface

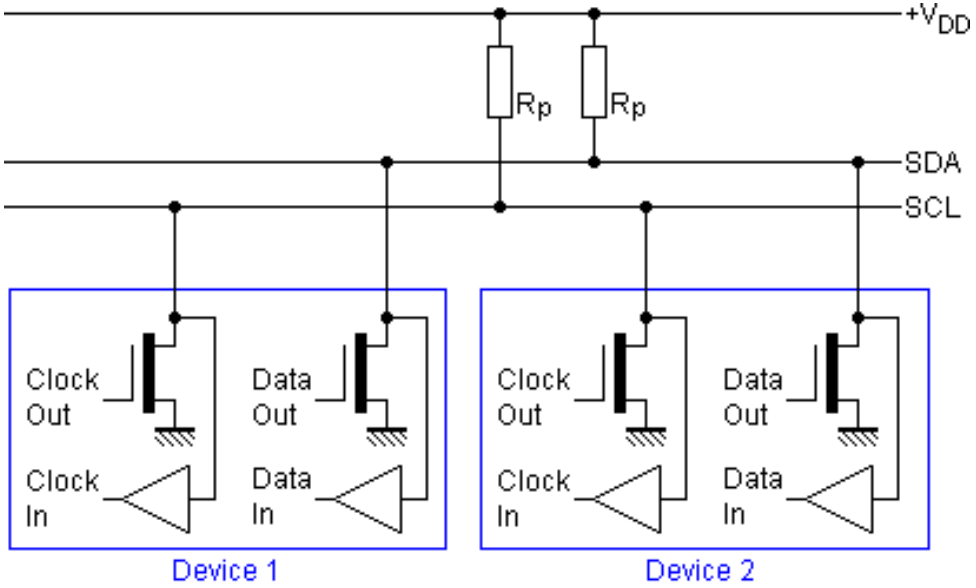
I²C protocol basics

- Inter Integrated Circuit → IIC → I²C
- Many devices share same bus
- **Multi-master, multi-slave protocol** → Open drain outputs (non-destructive arbitration, zero wins)

- Transmission rates: **100 kbps** (Standard mode), 400 kbps (Fast mode), 3.4 Mbps (High speed mode), 5 Mbps (Ultra-fast mode)
  Not supported by the STM32F401

- **Two wires** connection:
  - SDA (for data)
  - SCL (clock)
  - (+ common ground)

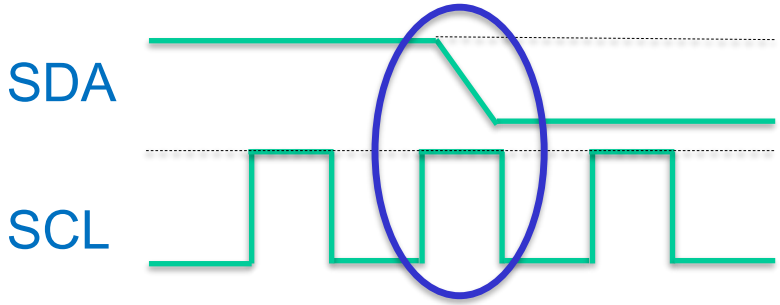
Both SDA and SCL are **bidirectional**



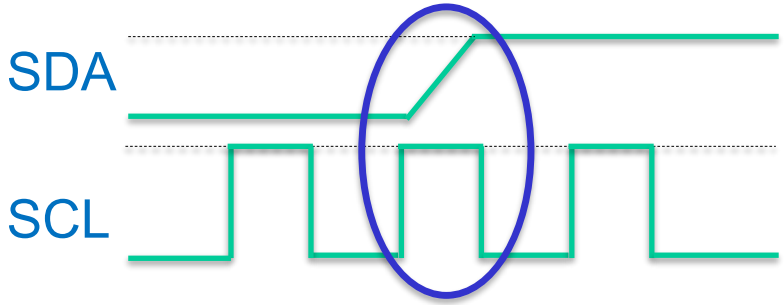
I²C protocol: START and STOP

When there's no trasmission, both SCL and SDA are kept «high».

START: transition from high to low of SDA, while SCL is kept high.



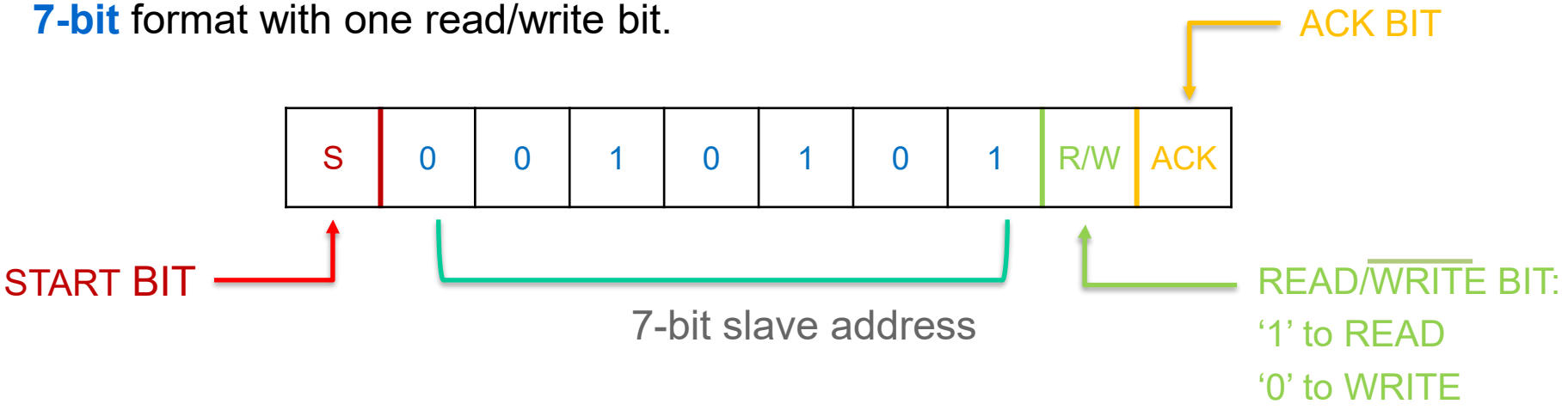
STOP: transition from low to high of SDA, while SCL is kept high.



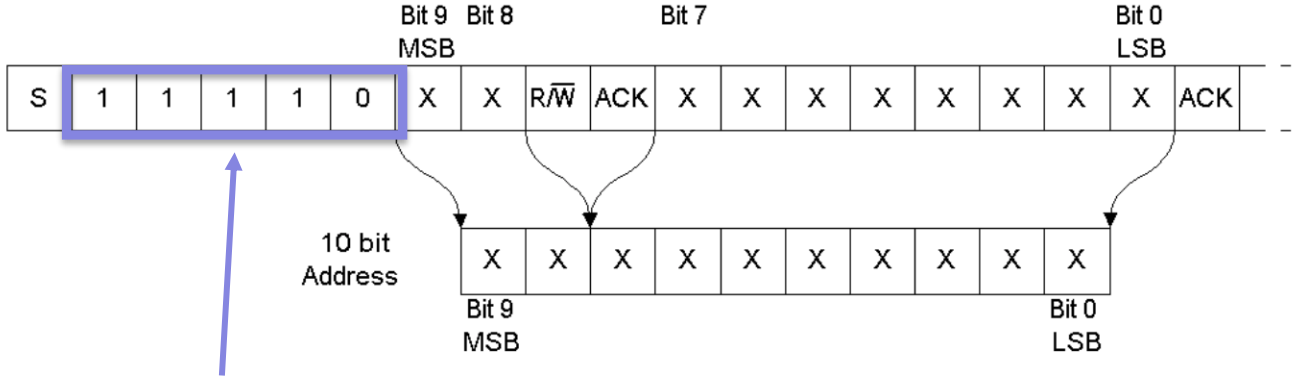
I²C protocol: addressing

Each device connected to the I2C bus has a **unique address**, in either of 2 formats:

- **7-bit** format with one read/write bit.



- **10-bit** format with one read/write bit



I²C protocol: data packets and speed

Data is packed in **bytes**, there's no limitation on the number of bytes.

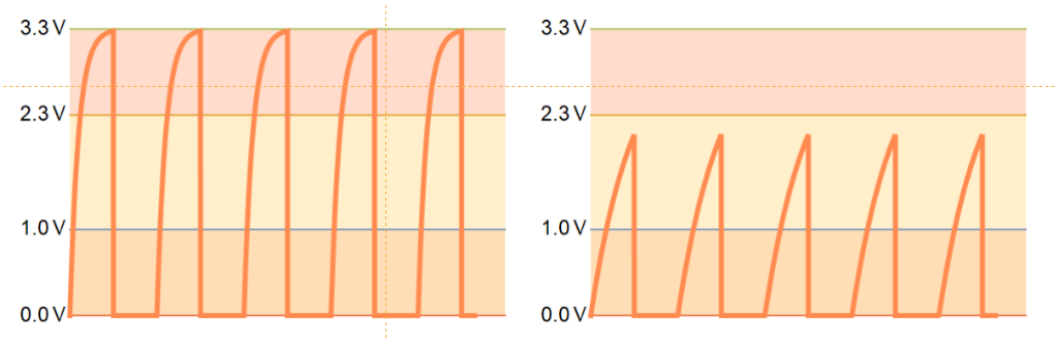
Acknowledge bit controls transmissions: if no ACK, communication stops:

- In **slave-receiver** mode, the slave necessarily needs to acknowledge the byte; otherwise the communication stops (e.g. if the address is wrong, no ACK is raised and communication halts)
- In **master-receiver** mode, the master deliberately doesn't ack the last byte in order to stop the communication.

High-Low transitions are **fast** (driven by an NMOS with low impedance).

Low-High transitions limited by circuit **RC**:

pull-up resistor value typ: 1 to 10 kΩ / bus capacitance typ. 10 to 100 pF

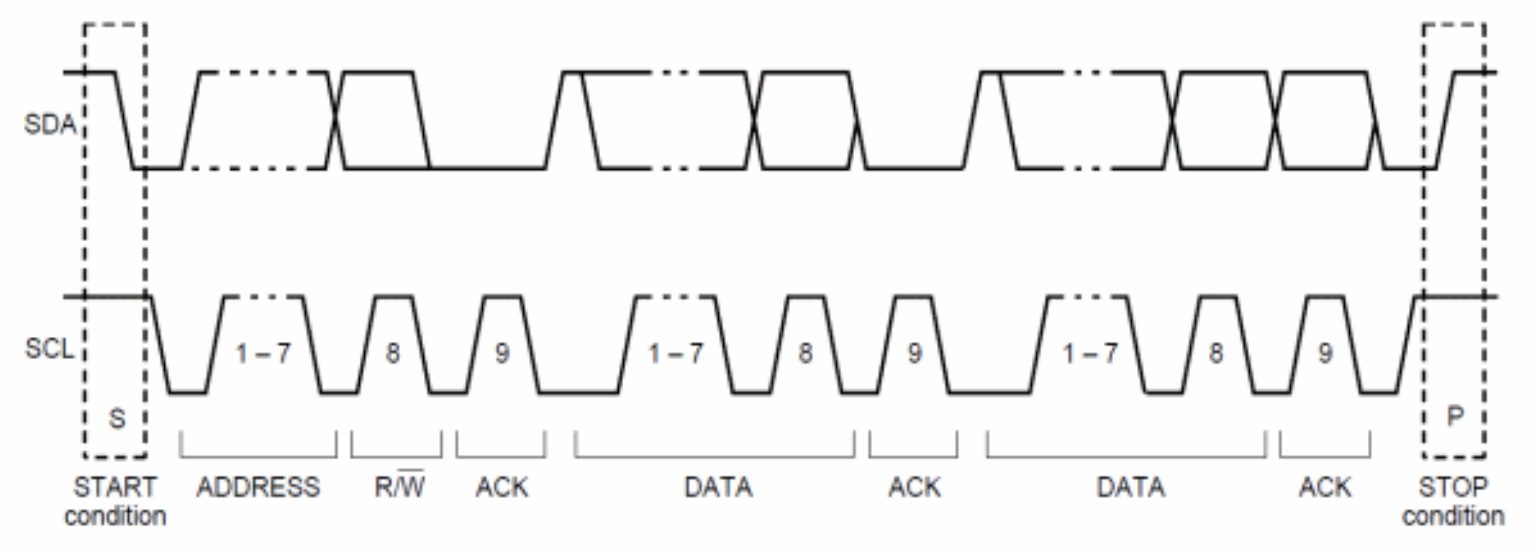


Normal operation! Too fast transitions!

I²C protocol: transmission example

Since a SDA transition during SCL high will bring to a START/STOP condition, the **data transfer** necessarly occurs **when SCL is low**.

Example of transmission:



ADC HAL functions

HAL functions for I²C:

HAL_StatusTypeDef **HAL_I2C_Master_Transmit**(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size, uint32_t Timeout)

HAL_StatusTypeDef **HAL_I2C_Master_Receive**(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint8_t *pData, uint16_t Size, uint32_t Timeout)

uint16_t DevAddress is the device address

uint8_t *pData is the data to be sent/received

uint16_t Size is the number of bytes to be sent/received

LM75/LM75B

- Band gap temperature sensor + 9 (B: 11) bit ADC
  LSB =0.5 °C (B: 0.125°C), range -55°C to 125°C
  8 bit integer temperature in °C (MSB sign bit)

  (two's complement digital data)

+ 3 bit decimal (LM75**B**)
+ 1 bit decimal (LM75)

Different board batches!

	MSByte								LSByte							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
LM75B	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	X	X	X	X	X
LM75	D8	D7	D6	D5	D4	D3	D2	D1	D0	X	X	X	X	X	X	X

- Temperature register (address 0x00) stores the converted temperature
- I^2C interface
- LM75/LM75B address: 1001 + 000 → 1001000 (7 bit address)

Project 1

Read the **temperature**
measured by the LM75
and send it to a remote terminal
every 1 second.

As a first step we will read only the MSB 8 bit.

Project hints

1. In CUBE configure the I2C in standard mode and enable the USART.

2. Generate the c code.

3. In the while loop read the MSByte of the temperature register using the I2C functions. When you declare the variables, watch out for the data types needed by the I2C functions.

4. Send the value to the PC using the USART interface and add a proper delay to repeat the loop every 1 s.

5. Do you need to do any conversions in order to obtain the value of temperature in °C?

Project 1b

Now we will modify the code
to read all 11 bits within an interrupt routine

Project hints

1. Modify the code to read both MSByte and LSByte of the temperature register.

2. Convert the values of MSByte and LSByte in a value of temperature with 3 decimal places.
   Hint: verify your conversion function with the example values reported on the datasheet in Table 10 (pp. 9-10)

3. Send the value to the PC using the USART interface using a timer to generate an interrupt every second.

Bug to be checked

If you only read once the temperature bytes, you might observe this bug.

In this case the temperature was decreasing, but for some reasons (it is up to you to find the reason) it passes from 26 °C to 26.875 °C instead of 25.875 °C.

```
Temperature: 26.250 °C
Temperature: 26.125 °C
Temperature: 26.000 °C
Temperature: 26.875 °C
Temperature: 25.875 °C
Temperature: 25.750 °C
Temperature: 25.625 °C
Temperature: 25.750 °C
Temperature: 25.500 °C
```

LM75 vs LM75B

LM75

If a conversion is in progress, it will be stopped and restarted after the read.

LM75B

When the LM75B is accessed the conversion in process is not interrupted (that is, the I2C-bus section is totally independent of the Sigma-Delta converter section) and accessing the LM75B continuously without waiting at least one conversion time between communications will not prevent the device from updating the Temp register with a new conversion result.

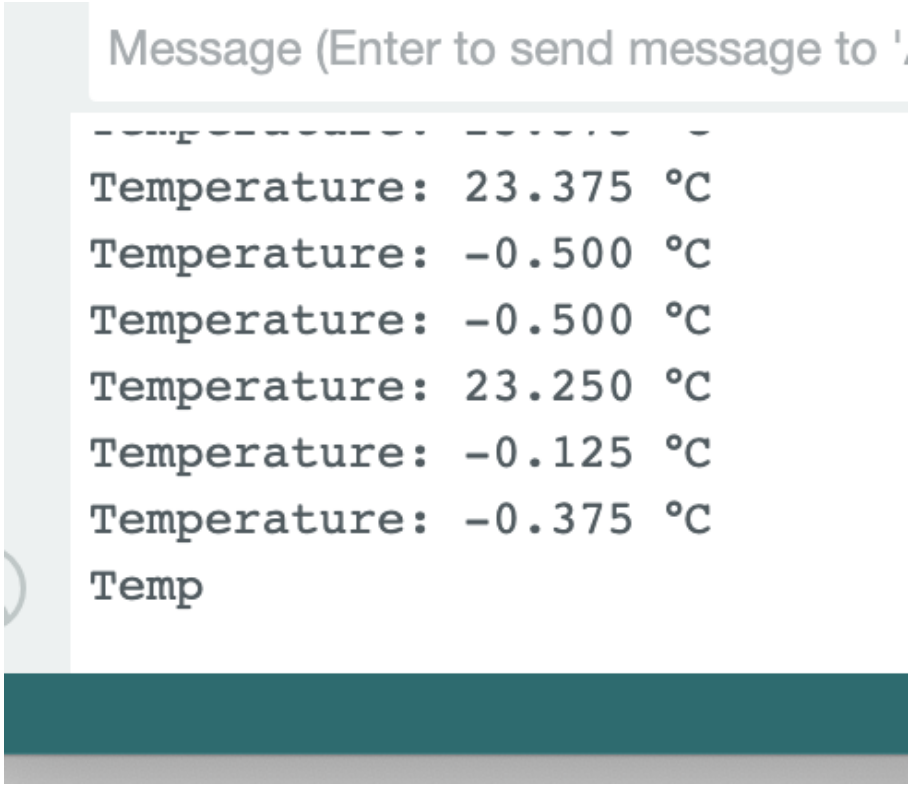
MEM_READ_DMA

```
// Callback for Timer Update Event [triggered every second]
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim == &htim1) {
//      Toggle_LED();
        // Start I2C DMA read from sensor
        HAL_I2C_Mem_Read_DMA(&hi2c1,
                LM75B_ADDR,  // Target device address: LM75B device address
                TEMP_REG_ADDR,  // Internal address: Temperature register address of LM75B
                I2C_MEMADD_SIZE_8BIT,  // The size of the memory address of the internal register
                therm_i2c_rx,  // Buffer to store the value
                2);  // Amount of data to be read: 2 bytes of data from the sensor
    }
}
```

- I2C_HandleTypeDef *hi2c --> ***&hi2c1***
- uint16_t DevAddress --> ***LM75B_ADDR***: this is the LM75B 7 bits device address which is extended to 8 bits, its value is initialized as ***(0b1001000 << 1)***
- uint16_t MemAddress --> ***TEMP_REG_ADDR***: it's the temperature register address, its value is ***0b00000000***
- uint16_t MemAddSize --> ***I2C_MEMADD_SIZE_8BIT***
- uint8_t *pData --> ***therm_i2c_rx***: it's the uint8_t data buffer of size 2 in which we are going to store the read data
- uint16_t Size --> ***2***: we are going to read two data bytes from the bus

Int8_t vs uint8_t

If the bytes read from the temperature register are stored in an int8_t array, the displayed values may appear incorrect. This occurs due to how data is handled during conversion—whether from int8_t to int16_t or from uint8_t to int16_t.



The compiler automatically extends the variable to accommodate the bit size of the new type. It fills the additional bits based on the original variable's type. If the starting variable is unsigned, the new bits are initialized to 0 (as there's no sign to consider). However, if the original variable is signed, the extra bits are filled to preserve the sign, using two's complement representation.

Int8_t vs uint8_t

Examples:

Positive number = 21

dec 21 -> uint8_t 00010101 -> uint16_t 0000000000010101

dec 21 -> int8_t 00010101 -> int16_t 0000000000010101

Negative number = -21

dec -21 -> int8_t 11101011 -> int16_t 1111111111101011

Essentially, when working with int variables in two's complement form, if the number is negative, any additional bits are initialized to 1.

Int8_t vs uint8_t: practical example

Let's make a practical example: T = 21.875 °C

The value saved in the temp register is 0001010111100000.

Then we save in int8_t temp[0] = 00010101 and in int8_t temp[1] = 11100000.

Then, since we want to merge this data in the int16_t temperature variable, what happens is that both temp[0] and temp[1] are converted from int8_t to int16_t, and now they are equal to:

int16_t temp[0] = 0000000000010101

int16_t temp[1] = 1111111111100000

Then temp[0] is bit shifted, and it becomes:

temp[0]<<8 = 0001010100000000

Then we make the or operation, and temp[0] | temp[1] = 1111111111100000

This is equal to -32, and -32/256 = -0.125. This is the reason of this bug. To solve it use uint8_t type for the temp array.